OS Project Description

Team : Barira Fatima and Safiyah Arif

31 July 2026

1 Background

Sudoku is considered to be a part of NP Complete problems (i.e NP (Nondeterministic Polynomial time) because if someone gives you a completed grid(even huge), you can quickly verify whether it follows all the rules (no repeating numbers in any row, column, or sub-grid) in polynomial time but if somebody gives a computer the same grid to solve (which it does by backtracking), it cannot (or atleast is not proven until) to be solved in polynomial time.

1.1 Idea behind our Project

We first solved a 9 x 9 sudoku sequentially and parallel-y (by using threads ofcourse). Then we did the same with 25 x 25 sudoko. It was found that 25 x 25 sudoko was faster solved by mutiple threads, however for small input size like 9 x 9, sequential is the better option.

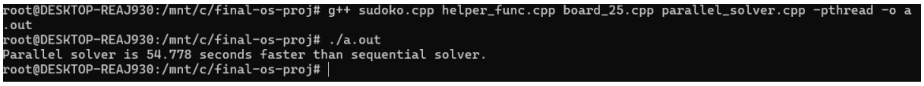
1.2 How was the backtracking implemented?

The worker finds the first empty cell, finds a list of valid numbers (let's say 1, 5, and 12), and writes down 1. They then move to the next cell and try to solve the entire rest of the board based on that initial choice of 1.

Now , if, after filling 77 cells, the worker hits a dead end where no numbers are valid, they realize their initial choice of 1 was wrong. They must manually erase all 77 cells, go all the way back to the first cell, change it to 5, and start the entire process over again.

1.3 How was the backtracking parallelized?

For each option, 1, 5 , 12, a thread makes the copy of the board, does this separately and when a thread solves it correctly , it takes lock to paste the solution onto the main board. Upon running it, one output for our board came out to be this:



Note: Sudoku is NP complete, and though for certain amount of input range, this might be useful, however currently we don't have any polynomial time solutions for it, so in a 1000 x 1000 sudoku, parallel and sequential will both fail miserably.

In git repo, Branch: feature 9 x 9 is contributed by Barira Branch: feature 25 x 25 is contributed by Safiyah.